Project 5 — Submission 2/2:DLH + Tools

High level Design Document

1. Introduction & System Overview..... 3

1.1 system Objectives.....3

1.2 Scope.....3

2. High Level Architecture..... 3

2.1 Architecture Rationale.....4

3. System Components..... 5

3.1 Frontend UI.....5

3.2 Falsk REST API.....5

3.3 Computerr Vision Engine (CV engine).....5

3.4 MySQL Database.....6

4. Exchanged Messages & Data Flow..... 6

4.1 API Endpoints.....6

4.2 Enrollement Flow: Message Sequence.....7

4.2 Enrollement Flow: Message Sequence.....7

4.3 Autentication Flow: Message Sequence.....8

4.4 Data Format Reference.....9

5. Tools & Technology Stack..... 10

5.1 Frontend Tools.....10

5.2 Backend Tools.....11

5.3 Developent & DevOps Tools.....11

6. Developemnt Phases..... 12

6.1 Phase Details.....13

7. Security Considerations..... 15

1. Introduction & System Overview

1.1 system Objectives

- Provides a ssecure, browser-accessible facial authentication mechanism
- Enroll isers by capturing and storing their facial faceprints in a database
- Authenticate users in real time by comparing a love camera frame against stored facepronts
- Operate wiythit clud dependencies, all processing hapens server-side on ocal infrastructure
- Remain lightweight and deployed on standard CPU hardware

1.2 Scope

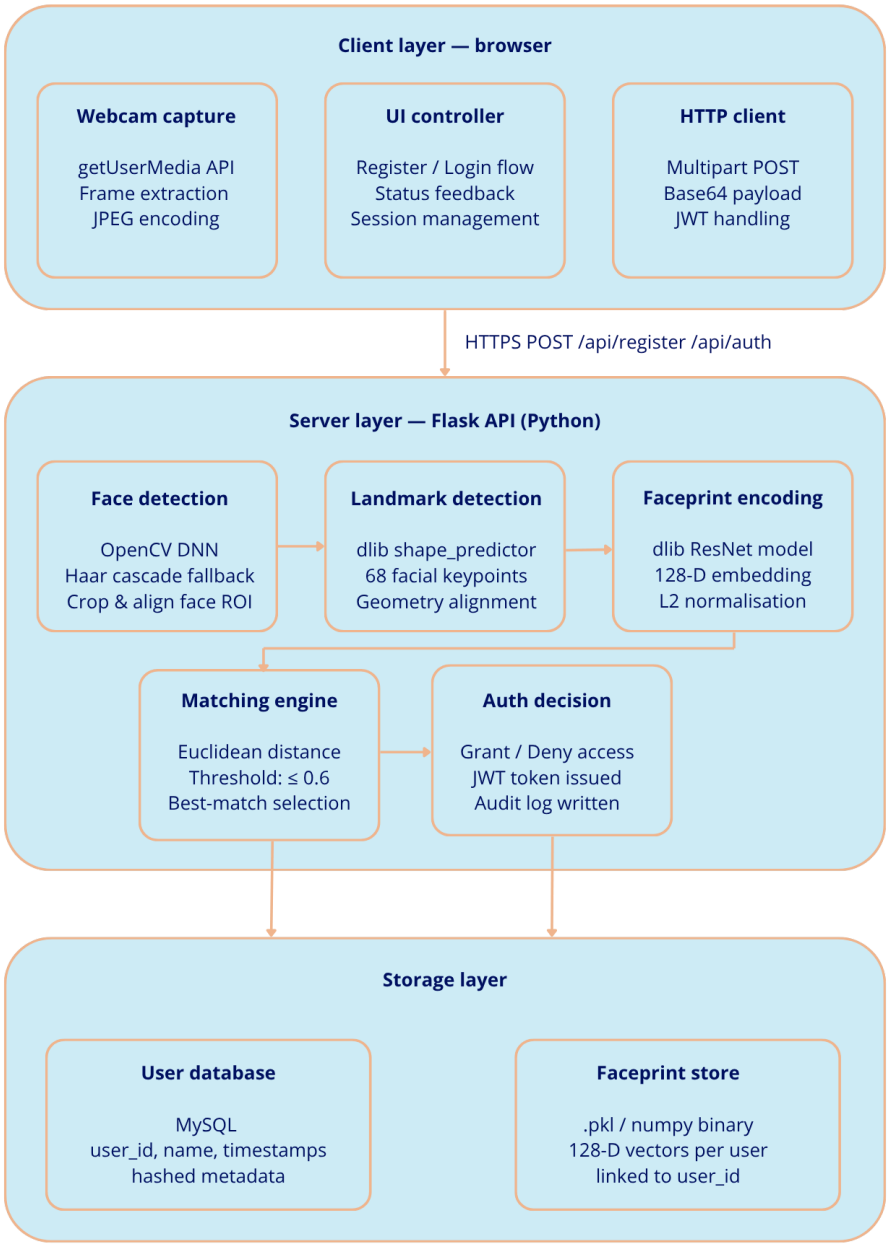
The system covers two main workflows:

- **Enrollement:** a new user registers by submitting their face via a webcam stream.
- **Authentication:** a retur,ing iser is verified by matchiong their live face against stored records.

Out of scope for this version: liveness detection, multi-factor authentication, and horizontal scaling.

2. High Level Architecture

The system follows a classic three-tier web architecture: a React rontend, a Flask REST API backend, and a SQLite presistence layer. All facial processing is contained within the backend service.



2.1 Architecture Rationale

The chosen architecture separates concerns clearly: the frontend handles user interaction and camera access, the backend performs all computer-vision computations, and the database stores persistent user data. This separation keeps teh facial processing logic reusable and independently testable.

3. System Components

The system is composed of ofur main components that communicate over defined interfaces

Component	Technology	Responsibility	Interfaces With
Fronted UI	React.js + WebRTC	Camera capture, UI rendering, API calls	Falsk API (HTTPS)
Flask API	Pythin 3.13 + Flask	Routes, session management, request routing	Frontend, CV Engine, DB
CV Engine	OpenCV + face_recognition	Detection, encoding, matching, liveness	Flask API only
Database	MySQL	Persist users, faceprints, sessions logs	Flask API only

3.1 Frontend UI

- Built with React.js using the WebRTC MediaDevices API for camera access
- Capture video frames at configurable intervals and converts them to Base64 JPEG
- Exposes two main viexws: enrollment page + authentication page
- Displays real-time feedback rteurned by teh backend
- Communicate exclusively wia HTTPS REST calls, noo direct DB or CV access

3.2 Falsk REST API

- Acts as the central orchestrator of all systel operations
- Exposes four primary endpoints
- Validates incoming requests before passing fraes to the CV Engine
- Manages JWT-based session tokens for authenticated users
- Writes authentication ebents and decisions to the audt log table.

3.3 Computerr Vision Engine (CV engine)

This is the core processing module, ilplemented as a Oython service class. It’s not directly accessible via the network, iy’s invoked only by the Flask API layer.

- **Face Detection:** uses OpenCV Haar Cascade or DNN module to locate and crop face regions from a frame.
- **Landmark Extraction:** maps 68 key facial points using dlib's shape predictor for alignment.
- **LBPH Encoding:** encodes the aligned face into a compact histogram-based faceprint vector.
- **Cosine Matching:** compares the query faceprint against all stored vectors and returns the closest match and distance score.

3.4 MySQL Database

The database stores four tables:

- Users: user_id, username, created_at, status
- Faceprints: faceprint_id, user_id(FK), vector (BLOB), captured_at
- Sessions: session_id, user_id (FK), jwt_token, issued_at, expires_ at
- Audit_logs: log_id, user_id, event_type, result, timestamp, ip_address

4. Exchanged Messages & Data Flow

4.1 API Endpoints

Method	Endpoint	Caller	Request Payload	Response
POST	/enroll	Frontend	{ "username": str, "frame": base64_str }	{ "status": "enrolled", "user_id": int }
POST	/authenticate	Frontend	{ "frame": base64_str }	{ "match": bool, "user": str, "score": float, "token": str }
POST	/logout	Frontend	{ "token": str }	{ "status": "logged_out" }

GET	/health	Monitoring	—	{ "status": "ok", "uptime": int }

4.2 Enrollement Flow: Message Sequence

Step	Action & Data Exchanged
1	User opens enrollment page → Browser requests camera permission via WebRTC getUserMedia()
2	User enters username and clicks 'Enroll' → Frontend captures a frame from the video stream and encodes it as Base64 JPEG
3	Frontend → Flask: POST /enroll { "username": "alice", "frame": "<base64>" }
4	Flask decodes Base64 frame → passes numpy array to CV Engine
5	CV Engine: detects face → extracts landmarks → generates 128-D LBPH faceprint vector
6	Flask: inserts user record into users table → stores faceprint BLOB in faceprints table → writes enrollment event to audit_logs
7	Flask → Frontend: 200 OK { "status": "enrolled", "user_id": 42 }
8	Frontend displays success confirmation to user

4.3 Autentication Flow: Message Sequence

Step	Action & Data Exchanged
1	User opens authentication page → Browser activates webcam stream
2	Frontend captures frame every 500ms and sends it to the backend
3	Frontend → Flask: POST /authenticate { "frame": "<base64>" }
4	Flask passes frame to CV Engine for detection and encoding
5	CV Engine generates query faceprint → Flask queries DB for all stored faceprint vectors
6	CV Engine computes cosine similarity between query vector and each stored vector → returns best match and distance score
7	Flask applies threshold (e.g., distance < 0.4 = match) → issues JWT token if accepted → writes result to audit_logs
8	Flask → Frontend: 200 OK { "match": true, "user": "alice", "score": 0.92, "token": "eyJ..." } OR { "match": false }
9	Frontend stores JWT token → redirects user to protected dashboard

4.4 Data Format Reference

All API communication uses JSON over HTTPS. Video frames are encoded as Base64 JPEG strings before transmission. Faceprint vectors are tored as binary BLOBs in MySQL and serialized/deserialized using numpy.

Frame Size	Each captured frame is resized to 640×480 pixels before Base64 encoding, producing approximately 60–80 KB per request. The backend enforces a 200 KB max payload limit.

Faceprint	128-dimensional float32 vector produced by the LBPH encoder. Stored as a numpy binary blob (~512 bytes). Multiple faceprints per user are supported for robustness.

JWT Token	HS256-signed token containing { user_id, username, iat, exp (24h) }. Frontend stores it in memory only (not localStorage). Sent in Authorization: Bearer header for protected routes.

5. Tools & Technology Stack

5.1 Frontend Tools

Tool	Version	Purpose
React.js	18.x	UI component framework and state management
WebRTC / getUserMedia	Browser API	Access device camera and capture video frames
Axios	1.x	HTTP client for REST API communication
TailwindCSS	3.x	Utility-first CSS for responsive UI styling

5.2 Backend Tools

Tool	Version	Purpose
Python	3.13	Primary backend language
Flask	3.x	Lightweight REST API framework
OpenCV	4.x	Face detection (Haar Cascade / DNN), image preprocessing

face_recognition (dlib)	1.3.x	128-D faceprint encoding via ResNet landmark pipeline
NumPy	1.26	Array operations and faceprint vector math
PyJWT	2.x	JWT token generation and validation
MySQL	Built-in	Lightweight relational DB for users, faceprints, sessions

5.3 Developent & DevOps Tools

Tool	Category	Purpose
Git / GitHub	Version Control	Source code management and collaboration
Docker	Containerization	Package backend with all CV dependencies for reproducible deployment
Postman	API Testing	Manual testing of REST endpoints during development
pytest	Unit Testing	Backend unit and integration tests (CV engine, API routes)
VS Code	IDE	Primary development environment

6. Developemnt Phases

This project is devided into five phases, progressing from environmental setup to final validation.

Phase	Name	Key Deliverables
1	**Environment Setup**	Git repo initialized, Python venv, Flask skeleton, React app bootstrapped, SQLite schema created
2	**CV Engine Development**	Face detection module, landmark extractor, LBPH encoder, cosine similarity matcher — all unit-tested in isolation
3	**Backend API Development**	/enroll and /authenticate endpoints, JWT middleware, database layer (users, faceprints, audit_logs), Postman collection
4	**Frontend Development**	Enrollment page, authentication page, webcam component, API integration, JWT session handling, UI feedback states
5	**Integration & Testing**	End-to-end testing of full pipeline, accuracy measurements, Docker containerization, final documentation

6.1 Phase Details

Phase 1: Environment Setup

- Initial Git repo with main/develop branch strategy
- Create Python virtual environment and install core dependencies (Flask, OpenCV, facce_recongnition)
- Bootstra React application using create-react-app or Vite
- Define and create the MySQL schema

Phase 2: CV Engine Developement

- Implement face detection using OpenCV Haar Cascade as primary detector.
- Integrate dlib 68-point landmark predictor for face alignment before encoding.
- Implement LBPH faceprint encoding and store it as numpy arrays.
- Write a cosine similarity matching function with configurable threshold.
- Write pytest unit tests for each CV module independently.

Phase 3: Backend API Development

- Implement Flask routes: POST /enroll, POST /authenticate, POST /logout, GET /health.
- Connect CV Engine to API layer via internal Python function calls.
- Implement JWT issuance and validation middleware.
- Implement database access layer (insert/query users and faceprints).
- Test all endpoints via Postman with real camera frames.

Phase 4 — Frontend Development

- Build Enrollment page with webcam preview, username input, and submit button.
- Build Authentication page with live camera feed and real-time result overlay.
- Implement Base64 frame capture and Axios POST integration.
- Handle JWT token in memory; route to protected dashboard on success.

Phase 5 — Integration & Testing

- Run full end-to-end tests: enroll a user, then authenticate from a different session.
- Measure recognition accuracy across different lighting conditions and distances.
- Tune cosine similarity threshold based on test results.
- Package the backend in a Docker container for reproducible deployment.
- Write final HLD and technical documentation.

7. Security Considerations

Although this system is a first implementation targeting simplicity, the following security measures are built into the design:

- HTTPS only: all API traffic is encrypted in transit; no plain HTTP endpoints are exposed.
- JWT expiration: tokens expire after 24 hours and are validated on every protected request.
- Payload validation: the backend rejects frames exceeding 200 KB and non-JPEG inputs.
- No biometric data leaves the server: faceprints are processed and stored server-side only.
- Audit logging: every authentication attempt (success or failure) is logged with timestamp and IP address.
- Threshold tuning: the cosine similarity threshold (default 0.4) is configurable to balance FAR (False Accept Rate) vs FRR (False Reject Rate).

Limitations acknowledged in this version: no liveness detection (susceptible to photo attacks), no rate limiting on the /authenticate endpoint, and no encryption at rest for faceprint BLOBs. These are candidates for Phase 2 improvements.